



From Legacy to Modern: A Framework for Diagnosing and Modernizing Data Pipelines for the Data-Driven Workforce

Lisette M. Kamper-Hinson



M.S. Computer Science · Wake Forest University Advisor: Dr. William H. Turkett · Committee: Dr. Ron P. Doyle · Dr. Jeffrey D. Camm

Abstract

Modernization of operational data pipelines frequently emphasizes analytical sophistication while neglecting structural governance, reproducibility, and decision usability. In manufacturing environments, this imbalance can exacerbate misalignment between reported and realized production capacity.

This thesis presents a structured, six-stage framework for modernizing legacy capacity planning pipelines while preserving embedded domain knowledge and minimizing technical debt. The framework spans: domain alignment, structural audit, prioritized modernization, governed modeling introduction, decision-aligned visualization design, and reproducible implementation with system-level health monitoring.

The approach is validated through a single-plant manufacturing case study using read-only operational data, incorporating capacity fill rate computation, short-horizon forecasting, scenario-based simulation, and outlier detection. Results demonstrate that modernization guided by explicit structural and governance constraints improves analytical reliability without increasing system fragility, and that modeling, visualization, and implementation standards are most effective when treated as integrated system design decisions.

Introduction

Modern infrastructure cannot compensate for brittle analytical workflows.

Data Science gained formal recognition in the early 2000s following Cleveland's (2001) call to bridge statistics and computing. Its reach now spans medicine, business, public policy, and manufacturing, yet structural practices remain uneven.

As the discipline matures, organizations face a critical paradox: they invest in modern cloud infrastructure while the internal analytical pipelines responsible for extraction, transformation, modeling, and reporting remain fragile and opaque. These pipelines, built incrementally over years, embed domain heuristics, undocumented transformations, and tightly coupled logic that resists adaptation.

The distinction between infrastructure modernization and pipeline modernization is essential. Infrastructure upgrades storage and scalability. Pipeline modernization addresses the analytical logic within systems: the structure of transformations, modularity of code, transparency of assumptions, and reproducibility of outputs.

Existing frameworks compound the problem. CRISP-DM, governs building new pipelines but offers no guidance for inheriting and transforming existing ones. Research identifies symptoms, hidden ML debt, pipeline quality failures, structural fragmentation, but no integrated, experimentally validated modernization methodology exists.

This thesis fills that gap through Design Science Research: iteratively building, testing, and validating a prescriptive modernization framework within a real manufacturing pipeline, evaluated against five process-metric categories to prevent metric recursion.

Mathematical Foundations

Capacity Fill Rate (CFR)

$$CFR_d = \frac{\text{Realized Output}_d}{\text{Effective Hours}_d}$$

Effective capacity = theoretical max – downtime – maintenance – partial-day interruptions (domain-expert-validated heuristics, preserved during modernization)

Holt's Linear Exponential Smoothing

Future capacity is estimated using Holt's linear exponential smoothing, which decomposes the time series into level and trend components:

$$L_t = \alpha C_t + (1 - \alpha)(L_{t-1} + T_{t-1})$$

$$T_t = \beta(L_t - L_{t-1}) + (1 - \beta)T_{t-1}$$

$$\text{Forecasts Generated as: } \hat{C}_{t+h} = L_t + \sum_{i=1}^h \phi^i T_t$$

News vendor Model: Optimal Capacity Buffer (Q*)

$$C(q, D) = c_{\text{cover}} \cdot \max(0, q - D) + c_{\text{under}} \cdot \max(0, D - q)$$

where C_u = underage cost (lost capacity) C_o = overage cost (idle capacity)

Under this formulation, the optimal planned capacity is given by the quantile:

$$q^* = F^{-1}\left(\frac{c_{\text{cover}}}{c_{\text{cover}} + c_{\text{under}}}\right)$$

Used in the Scenario Simulation dashboard to set statistically optimal capacity buffer thresholds, balancing the cost of under-producing against the cost of idle capacity.

The Data Pipeline Modernization Framework

PHASE 1: UNDERSTANDING

Domain Expert Meeting

Establish pipeline context, organizational goals, end users, constraints and data environment

Step 1

Structural Audit and Decomposition

Visualize data flow, document inputs, outputs, data states, dependencies and open questions

Step 2

Functional Evaluation and Prioritization

Classify components: preserve, refactor, redesign
Evaluate impact, risk, maintainability and alignment
Produce modernization roadmap

Step 3

Iterative: domain expert validation required before advancing. If misaligned, revisit Step 1.

PHASE 2: ANALYSIS AND DESIGN

Modeling Considerations

Evaluate candidate techniques, audit data, compare baselines, assess interpretability and technical debt

Step 4

Applied per pipeline component, not globally. May be revisited multiple times across the pipeline.

Visualization and Communication Design

Anchor each visual to a decision context, preserve operational knowledge and encode uncertainty

Step 5

Applied per decision-facing output. May be revisited as new outputs are exposed or analytical behavior changes.

PHASE 3: IMPLEMENTATION

Standardize and Validate

Reproducible execution, system-level health view, validate analytical, communicative and operational intent

Step 6

Application of the Framework

PRESERVED (domain knowledge intact)

REFACTORED (same logic, governed structure)

REDESIGNED (new analytical approach)

1. Ingestion & Temporal Normalization

10_compute_cfr_daily.py

- Production logs, downtime, changeover records
- Daily granularity normalized at ingestion
- Schema validation added
- Hard-coded file paths externalized to config

Preserved: Daily aggregation logic: preserved from legacy Excel process

2. CFR Computation

10_compute_cfr_daily.py

- CFR = Realized Output / Effective Hours
- Downtime, changeover, sanitation adjustments
- Domain buffer heuristics formally documented
- Gallon-equivalent normalization layer added

Preserved: Core CFR formula and all buffer heuristics: preserved exactly

3. Capacity Aggregation

20_compute_capacity_daily.py

- Time-indexed capacity series by Barrel line
- Descriptive aggregation isolated from modeling
- Explicit stage boundary enforced
- All parameters externalized

Preserved: Aggregation logic and thresholds: preserved from legacy spreadsheet

4. Governed Forecasting

40_forecast_capacity.py

- Holt's exponential smoothing (interpretable)
- News vendor optimization for plan targets
- Training-validation separation enforced
- Historical vs. forecast explicitly partitioned

5. CFR Ranking

50_rank_cfr.py

- Cross-sectional performance positions per Barrel
- Fully isolated from forecasting stage
- No predictive artifacts in descriptive output
- Consistent with legacy ranking intent

Preserved: Ranking intent and Barrels grouping logic: preserved from legacy RCCP

6. Streamlit Visualization

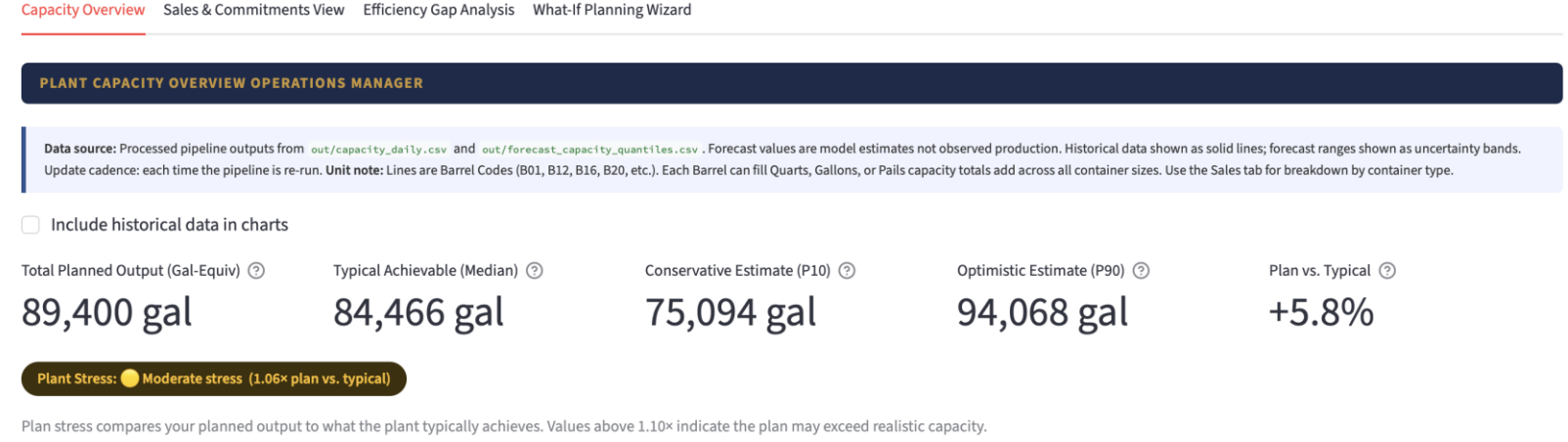
app.py

- 4-tab decision-support application
- Observed vs. forecast visually distinguished
- Stable axis encodings, Barrel breakdown
- Download: CSV plan + TXT summary

Streamlit Dashboard Suite

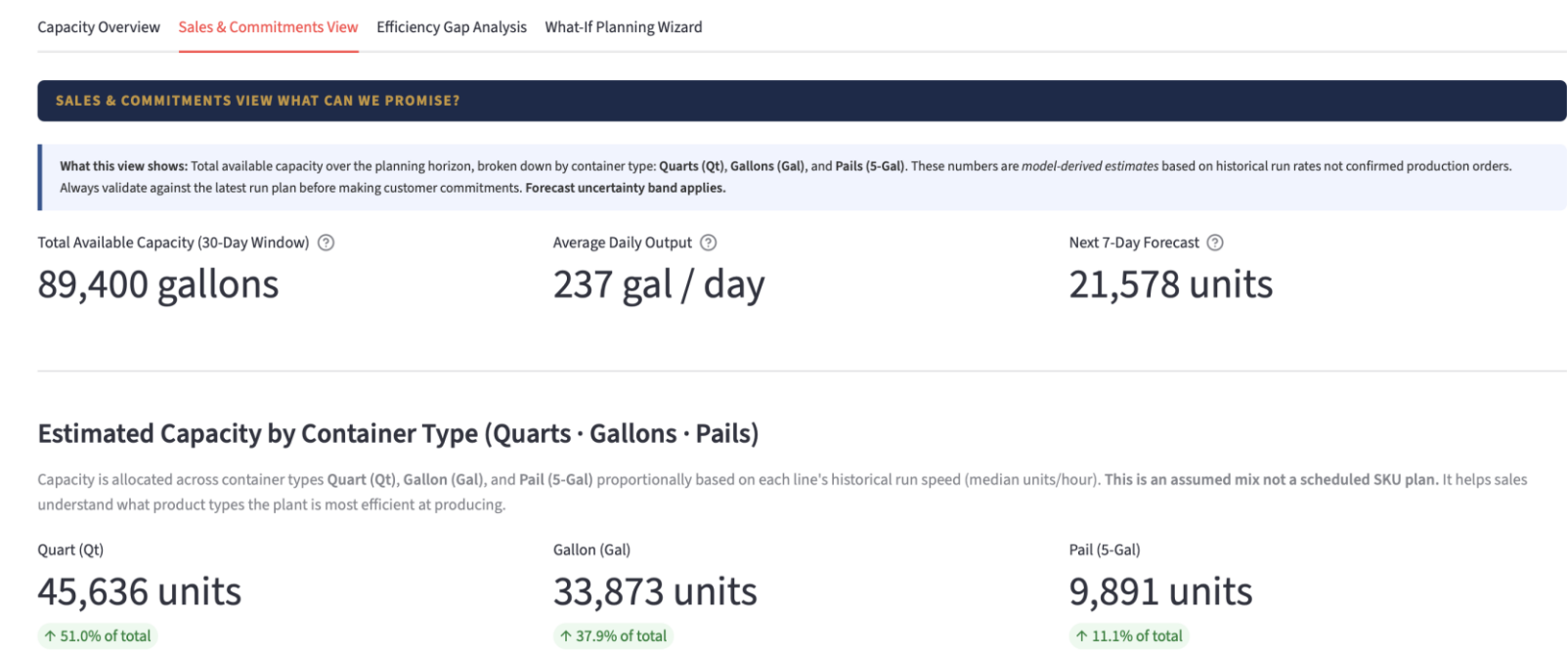
01 Capacity Overview

Your daily operations view. Shows the full planned output picture, how it compares to what the plant can realistically achieve, and which weeks look heavy or light.



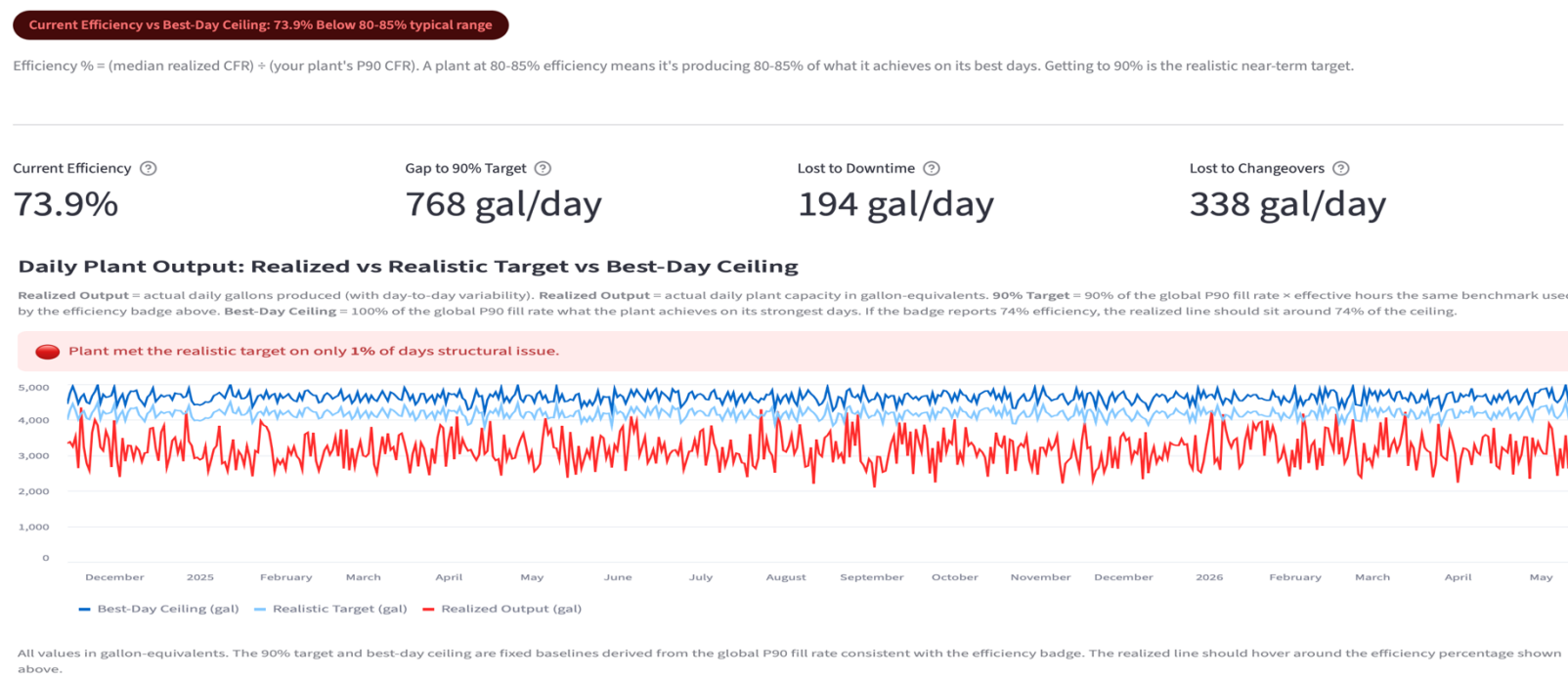
02 Sales & Commitments View

Answers the question: "What can we safely promise to customers?" Shows capacity broken down by container type (Quart / Gallon / Pail) with low / typical / high ranges.



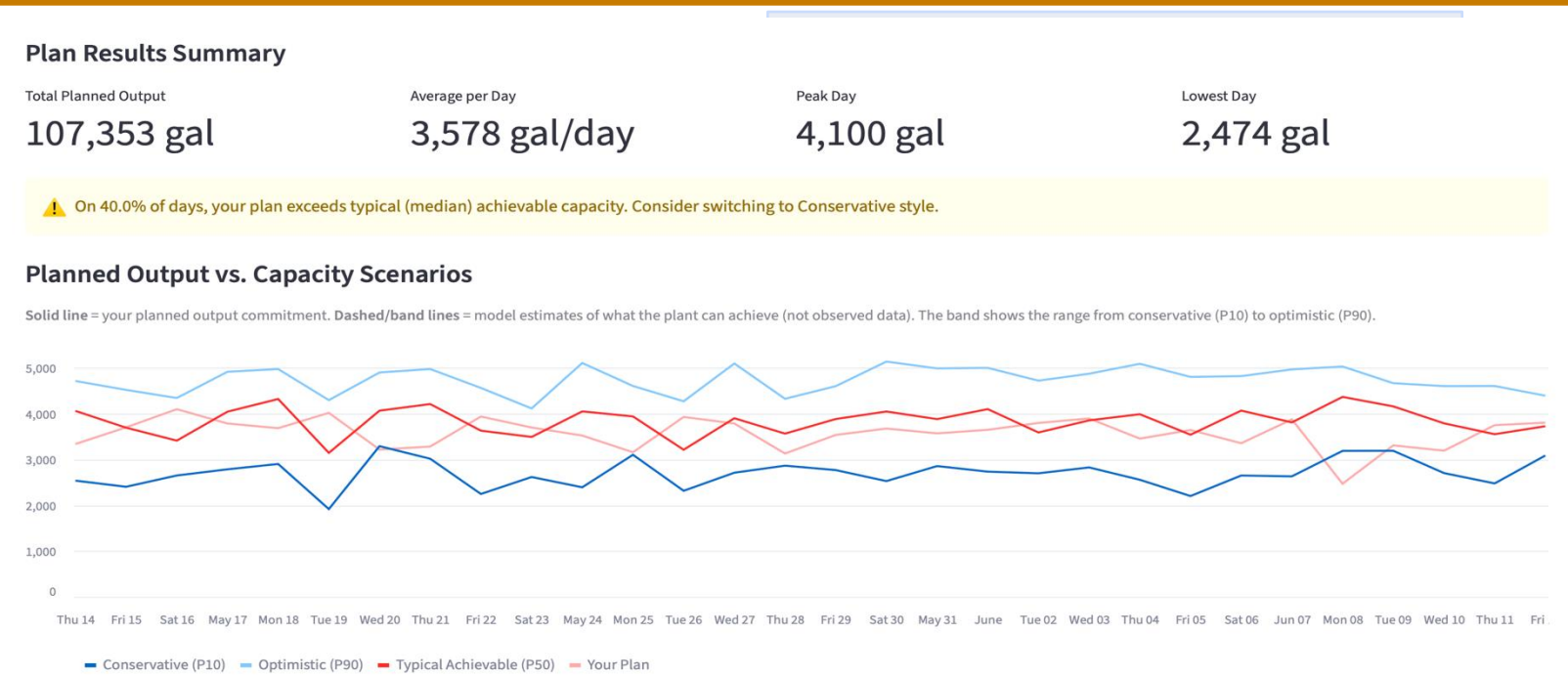
03 Efficiency Gap Analysis

Shows where your capacity is being lost, downtime, changeovers, or run rate, and what closing the gap to 90% efficiency would mean in real gallons.



04 What-If View

Allows you model a custom plan by adjusting your shift schedule, downtime, changeovers, and efficiency assumptions.



Conclusion

Pipeline modernization is a distinct and necessary area of methodological development within the broader Data Science discipline.

This thesis develops and validates a six-stage framework for modernizing legacy data pipelines, addressing the absence of a prescriptive methodology in contemporary data science practice. Applied within a real manufacturing system, the framework demonstrates that modernization is fundamentally architectural, structural clarity, modeling isolation, and configuration externalization improve maintainability, interpretability, and reproducibility without wholesale replacement. The result is a governed analytical asset that preserves embedded domain knowledge while sustaining integrity under real-world constraints.